MICROPROCESSOR CONTROL AND INTERFACING

Registers

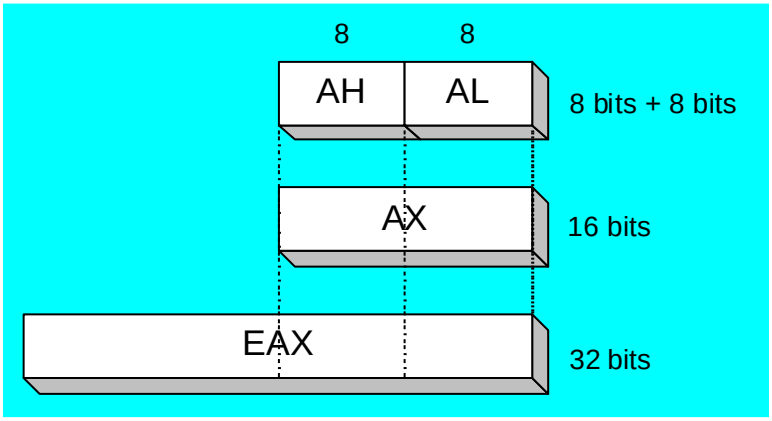
Register: High-speed storage locations in the CPU

Types of registers:

1. Data, or general-purpose (EAX, EBX, ECX, EDX)
2. Pointer (EBP, ESP, EIP)
3. Index (ESI, EDI)
4. Segment (CS, DS, SS, ES, FS, GS)
5. Flags (EFLAGS)

Accessing a Register

- Taking the EAX register as an example:
  - "E" in eax means extended
  - EAX = 32 bits
  - AX = lower 16 bits of eax
  - AH = upper 8 bits of ax
  - AL = lower 8 bits of ax



- MOV instruction moves a value into a register
  - mov eax, 3 eax = 00000003h
  - mov ax, 3 eax = ????0003h
  - mov ah, 3 eax = ????03??h

➔ Use 8-bit name, 16-bit name, or 32-bit name is Applied to EAX, EBX, ECX, and EDX.

32-bit	16-bit	8-bit (high)	8-bit (low)
EAX	AX	AH	AL
EBX	BX	BH	BL
ECX	CX	CH	CL
EDX	DX	DH	DL

➔ Some registers have only a 16-bit name for their lower half.

32-bit	16-bit
ESI	SI
EDI	DI
EBP	BP
ESP	SP

Data Registers: For arithmetic & data movement

- eax – accumulator
  - Used in multiplying & dividing
  - Preferred because generates shortest machine code
- ebx – base
  - Holds the address of a procedure or variable
  - esi, edi, ebp also have the same ability
- ecx – counter
  - Counter for repeating or looping instructions
- edx – data
  - Used in multiplying & dividing
  - Also in I/O operations

Pointer Registers: Contain data & instruction addresses

- ebp – base (or frame) pointer
  - Contains an address that is used to locate variables passed on the stack
  - For example, variables in function calls
- esp – stack pointer
  - Contains the address of the top (or bottom) of the stack
- eip – instruction pointer

- Contains address of next instruction to be executed
- Also called the “program counter” (pc)

Index Registers: Contains data offsets (for arrays)

- esi – source index
  - Points to source string in string movement instructions (memory location addressed by ds)
- edi – destination index
  - Points to destination string in string movement instructions (memory location addressed by es)

(E)FLAGS Register: Status flags – arithmetic & logic outcomes

- CF – carry flag
  - Result of unsigned arithmetic operation too large
- OF – overflow flag
  - Result of signed arithmetic operation too large
- SF – sign flag
  - 1 = negative result; 0 = positive result
- ZF – zero flag
  - 1 = zero result; 0 = nonzero result

Part II: Defining Data

Intrinsic Data Types:

1. BYTE, SBYTE
   8-bit unsigned integer; 8-bit signed integer
2. WORD, SWORD
   16-bit unsigned & signed integer
3. DWORD, SDWORD
   32-bit unsigned & signed integer
4. QWORD
   64-bit integer
5. TBYTE
   80-bit integer
6. REAL4
   4-byte IEEE short real
7. REAL8
   8-byte IEEE long real
8. REAL10
   10-byte IEEE extended real

- A data definition statement sets aside storage in memory for a variable.
- May optionally assign a name (label) to the data

Syntax:

[*name*] *directive initializer* [,*initializer*]

All initializers become binary data in memory

1. Defining BYTE and SBYTE Data

a) Defining Bytes

A variable name is a data label that implies an offset (an address). The offset of a variable is the distance from the beginning of the data segment to the first byte of the variable

Examples:

```
value1 BYTE 'A'          ; character constant
value2 BYTE 0            ; smallest unsigned byte

value3 BYTE 255          ; largest unsigned byte
value4 SBYTE -128        ; smallest signed byte
value5 SBYTE +127        ; largest signed byte
value6 BYTE ?            ; uninitialized byte
```

Examples that use multiple initializers:

```
list1   BYTE 10,20,30,40
list2   BYTE 10,20,30,40
        BYTE 50,60,70,80
        BYTE 81,82,83,84
list3   BYTE ?,32,41h,00100010b
list4   BYTE 0Ah,20h,'A',22h
```

b) Defining Strings

A string is implemented as an array of characters

a. For convenience, it is usually enclosed in quotation marks
b. It usually has a null byte at the end

```
.data
Var1 byte "ABC"
Var2 byte "DEFG"
```

If **Var1** at the beginning of the data segment. The offset of Var1 is 0 = the offset of **'A'**

The offset of **'B'** is 1

The offset of **'C'** is 2

The offset of **Var2** is 3

Examples:

```
str1    BYTE "Enter your name",0
str2    BYTE 'Error: halting program',0
str3    BYTE 'A','E','I','O','U'
greeting        BYTE "Welcome to the Encryption Demo program"
                BYTE "created by Kip Irvine.",0
```

To continue a single string across multiple lines, end each line with a comma:

Examples:

```
menu  BYTE "Checking Account",0dh,0ah,0dh,0ah,
        "1. Create a new account",0dh,0ah,
        "2. Open an existing account",0dh,0ah,
        "3. Credit the account",0dh,0ah,
        "4. Debit the account",0dh,0ah,
        "5. Exit",0ah,0ah,
        "Choice> ",0
```

c) Using the DUP Operator

- Use DUP to allocate (create space for) an array or string.
- Counter and argument must be constants or constant expressions

Examples:

```
var1 BYTE 20 DUP(0)             ; 20 bytes, all equal to zero
var2 BYTE 20 DUP(?)             ; 20 bytes, uninitialized
var3 BYTE 4 DUP("STACK")        ; 20 bytes: "STACKSTACKSTACKSTACK"
```

2. Defining WORD and SWORD Data

a. Define storage for signed or unsigned 16-bit integers
b. or double characters
c. single value or multiple values

Examples:

```
word1 WORD   65535      ; largest unsigned value
word2 SWORD –32768      ; smallest signed value
word3 WORD   ?          ; uninitialized, unsigned
word4 WORD "AB"         ; double characters
myList WORD 1,2,3,4,5   ; array of words
```

array WORD 5 DUP(?) ; uninitialized array

3. Defining DWORD and SDWORD Data

Storage definitions for signed and unsigned 32-bit integers.

Examples:
```
val1 DWORD 12345678h          ; unsigned
val2 SDWORD –2147483648       ; signed
val3 DWORD 20 DUP(?)          ; unsigned array
val4 SDWORD –3,–2,–1,0,1      ; signed array
```

4. Defining QWORD, TBYTE

Storage definitions for quadwords & tenbyte values.

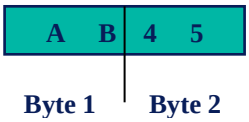
Examples:
```
quad1 QWORD  1234567812345678h
val1  TBYTE  100000000123456789Ah
```

How Data is Stored?

Var1 byte 5



Var2 word 45ABh



Part III: Data Transfer Instructions

1. MOV Instruction

The MOV instruction transfers the content of the source operand to the destination operand. The destination operand's contents change but the source operand is unchanged.

Syntax: MOV destination, source

- Both operands must be of the same size.
- An operand can be either direct or indirect
  Direct operands:
  - immediate (imm) (constant or constant expression)
  - register (reg)
  - memory variable (mem) (with displacement)

Indirect operands: are used for indirect addressing.

- The type of an operand is given by its size (byte, word, doubleword, …)

Some Restrictions on MOV Instruction

- imm cannot be the destination operand
- Instruction Pointer (IP) cannot be an operand
- source and destination cannot both be mem (direct memory-to-memory data transfer is forbidden!)

Types of MOV instructions:

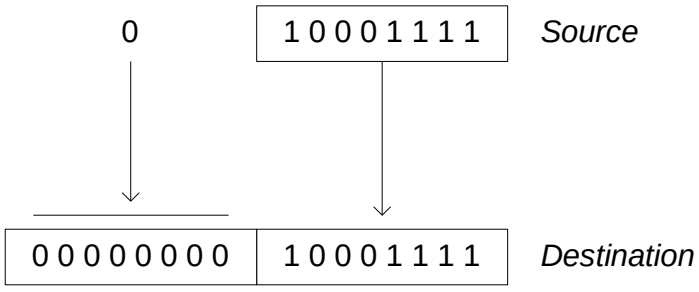
MOV Register , Register
MOV Register , Memory
MOV Register , Immediate
MOV Memory , Immediate
MOV Memory , Register

Examples of MOV usage:

```
mov bh, 255              ; 8-bit operands
mov al, 256              ; error: source too large
mov bx, AwordVar         ; 16-bit operands
mov bx, AbyteVar         ; error: size mismatch
mov edx, AdoublewordVar  ; 32-bit operands
mov cx, bl               ; error: operand not of same size
mov wordVar1, wordVar2   ; error: mem-to-mem
```

1.1 MOV with Zero Extension

When you copy a smaller value into a larger destination, the MOVZX instruction fills (extends) the upper half of the destination with zeros.

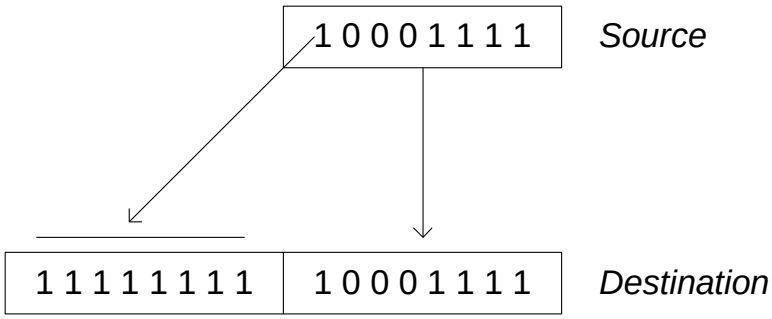



```
mov bl,10001111b
movzx ax,bl          ; zero-extension
                     ; ax = 0000 0000 1000 1111b
```

➔The destination must be a register.

1.2 MOV with Sign Extension

The MOVSX instruction fills the upper half of the destination with a copy of the source operand's sign bit.




```
mov bl,10001111b
movsx ax,bl         ; sign extension
                    ;ax = 1111 1111 1000 1111b
mov bl,01111111b
movsx ax,bl         ; sign extension
                    ;ax = 0000 0000 0111 1111b
```

2. XCHG Instruction

XCHG exchanges the values of two operands. At least one operand must be a register. No immediate operands are permitted.

Syntax: XCHG destination, source

- Where both operands must be of the same size.
- Neither operand can be a constant or immediate data.
- Both operands cannot be memory locations.

Examples:

```
.data
        var1 WORD 1000h
        var2 WORD 2000h
.code
        xchg ax,bx          ; exchange 16-bit regs
        xchg ah,al          ; exchange 8-bit regs
```

```
xchg var1,bx          ; exchange mem, reg
xchg eax,ebx          ; exchange 32-bit regs
xchg var1,var2        ; error: two memory operands
```

To exchange the content of word1 and word2, we have to do:

```
mov ax,word1
xchg word2,ax
mov word1,ax
```

Example: Adding and Subtracting Integers

```
TITLE Add and Subtract          (AddSub.asm)

; This program adds and subtracts 32-bit integers.

INCLUDE Irvine32.inc
.code
main PROC
        mov eax,10000h          ; EAX = 10000h
        add eax,40000h          ; EAX = 50000h
        sub eax,20000h          ; EAX = 30000h
        call DumpRegs           ; display register values in hexadecimal.
        exit
main ENDP
END main
```

Program Output, showing registers and flags:

```
EAX=00030000   EBX=7FFDF000   ECX=00000101   EDX=FFFFFFFF
ESI=00000000   EDI=00000000   EBP=0012FFF0   ESP=0012FFC4
EIP=00401024   EFL=00000206   CF=0  SF=0  ZF=0  OF=0
```

Problem 1: Write an assembly program to:

1. **Define the variables**

val1=1000h of type word
val2=2000h of type word
array1= 10h, 20h, 30h, 40h, 50h of type byte
array2=100h, 200h, 300h of type word
array3=1000h, 2000h of type dword

2. Using MOV, MOVZX and MOVSX instructions

Moving bx to eax
Moving bl to edx
Moving bl to cx

Where **bx=a69bh**

3. Using XCHG instruction to exchange the variables val1 & val2

Solution:

```
INCLUDE Irvine32.inc
.data
val1  WORD 1000h
val2  WORD 2000h

array1  BYTE  10h,20h,30h,40h,50h
array2  WORD  100h,200h,300h
array3  DWORD 10000h,20000h

.code
main PROC

;  MOVZX
        mov    bx,0A69Bh
        movzx  eax,bx          ; EAX = 0000A69Bh
        movzx  edx,bl          ; EDX = 0000009Bh
        movzx  cx,bl           ; CX  = 009Bh
        call dumpregs

;  MOVSX
        mov   bx,0A69Bh
        movsx eax,bx           ; EAX = FFFFA69Bh
        movsx edx,bl           ; EDX = FFFFFF9Bh
        movsx cx,bl            ; CX  = FF9Bh
        call dumpregs

;  Memory-to-memory exchange:
        mov  ax,val1           ; AX = 1000h
        xchg ax,val2           ; AX = 2000h, val2 = 1000h
        mov  val1,ax           ; val1 = 2000h

        exit
main ENDP
END main
```

Program Output

